FastAPI — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction and Installation

02 First API

03 Path and Query Parameters

04 Request Bodies with Pydantic

05 Response Models and Status Codes

06 Handling Errors

Intermediate (6 topics)

07 Validation and Field Constraints

08 Dependency Injection

09 Databases with SQLAlchemy

10 Routers and Project Structure

11 Async Endpoints

12 Background Tasks

Advanced (6 topics)

13 Authentication: OAuth2 and JWT

14 Middleware and CORS

15 File Upload and Streaming

16 WebSockets

17 Testing

18 Settings and Configuration

Master (4 topics)

19	Performance and Caching
20	Docker and Deployment
21	Serving ML Models
22	Real Project: Blog / Task API with Auth

Beginner Level

01. Introduction and Installation

DEFINITION

FastAPI is a modern, fast Python web framework for APIs built on type hints, Pydantic and Starlette, with automatic docs.

SYNTAX

```
fastapi dev main.py
```

EXAMPLE

```
pip install "fastapi[standard]"
fastapi dev main.py
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A fast delivery counter that checks every order form automatically; FastAPI validates requests using type hints.

தமிழ்

ஒவ்வொரு ஆர்டர் படிவத்தையும் தானாகச் சரிபார்க்கும் வேகமான டெலிவரி கவுண்டர் போல FastAPI type hints கொண்டு requests-ஐச் சரிபார்க்கிறது.

PROS

- Very fast (async)
- Automatic docs
- Less code, fewer bugs
- Editor autocomplete

CONS

- Younger ecosystem than Django
- Async mistakes are easy
- Needs separate ORM/admin choices

ADVANCED / NEW FEATURES

- `fastapi[standard]` and `fastapi dev/run` CLI
- Pydantic v2 (Rust core)
- Lifespan events
- `Annotated` dependency style

02. First API

DEFINITION

Create an app and path operations with decorators; docs at `/docs` (Swagger) and `/redoc`.

SYNTAX

```
@app.get('/path')
```

EXAMPLE

```
from fastapi import FastAPI
app = FastAPI()

@app.get('/')
def root():
    return {'message': 'Hello FastAPI'}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop's front desk: each request type has a counter. Path operations connect URLs to Python functions.

தமிழ்

கடை வரவேற்பு மேசையில் ஒவ்வொரு கோரிக்கை வகைக்கும் கவுண்டர் போல path operations URL-ஐ Python functions-உடன் இணைக்கின்றன.

PROS

- Tiny code
- Swagger UI at /docs

CONS

- Everything in one file gets messy

ADVANCED / NEW FEATURES

- ReDoc and OpenAPI 3.1
- Typed responses

03. Path and Query Parameters

DEFINITION

Typed parameters are validated automatically.

SYNTAX

```
def f(item_id: int, q: str | None = None):
```

EXAMPLE

```python
@app.get('/items/{item_id}')
def item(item_id: int, q: str | None = None):
    return {'id': item_id, 'q': q}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A parcel label gives the number (path) and delivery notes (query); types are checked automatically.

தமிழ்

பார்சல் லேபிளில் எண் (path), குறிப்புகள் (query) இருக்கும்; வகைகள் தானாகச் சரிபார்க்கப்படும்.

PROS

- Automatic conversion/validation
- Clear error messages

CONS

- Complex constraints need `Field/Query`

ADVANCED / NEW FEATURES

- `Annotated[...]` style
- Query parameter models

04. Request Bodies with Pydantic

DEFINITION

Pydantic models validate JSON bodies and generate schemas.

SYNTAX

```python
class Item(BaseModel):
    name: str
```

EXAMPLE

```python
from pydantic import BaseModel
class Item(BaseModel):
    name: str
    price: float

@app.post('/items')
def create(item: Item):
    return item
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

An application form with mandatory boxes; Pydantic models validate JSON bodies.

தமிழ்

கட்டாயக் கட்டங்கள் உள்ள விண்ணப்பப் படிவம் போல Pydantic models JSON body-ஐச் சரிபார்க்கின்றன.

PROS

- Strict validation
- Auto JSON schema

CONS

- Duplicate input/output models

ADVANCED / NEW FEATURES

- Pydantic v2: much faster, `model_validate`, `computed_field`

05. Response Models and Status Codes

DEFINITION

Control output shape with `response_model` and set codes with `status_code`.

SYNTAX

```
@app.post('/x', response_model=Out, status_code=201)
```

EXAMPLE

```
@app.post('/users', response_model=UserOut, status_code=201)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A bill that shows only what the customer needs, not the shop's internal cost price; `response_model` filters output.

தமிழ்

வாடிக்கையாளருக்குத் தேவையானதை மட்டும் காட்டும் பில் போல `response_model` வெளியீட்டை வடிகட்டுகிறது.

PROS

- Prevents leaking fields
- Documented responses

CONS

- Extra model classes

ADVANCED / NEW FEATURES

- Typed return annotations can replace `response_model`

06. Handling Errors

DEFINITION

Raise `HTTPException` with status and detail; add custom exception handlers.

SYNTAX

```
raise HTTPException(404, 'not found')
```

EXAMPLE

```
from fastapi import HTTPException
raise HTTPException(status_code=404, detail='Item not found')
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A polite 'item not available' slip instead of silence; HTTP exceptions return clear errors.

தமிழ்

மௌனத்துக்குப் பதிலாக 'பொருள் இல்லை' என்ற மரியாதையான சீட்டு போல HTTP exceptions தெளிவான பிழைகளைத் தருகின்றன.

PROS

- Standard errors
- Custom handlers

CONS

- Easy to leak internals

ADVANCED / NEW FEATURES

- ProblemDetails style handlers via libraries

Intermediate Level

07. Validation and Field Constraints

DEFINITION

`Field`, `Query`, `Path` add rules like `min_length`, `gt`, regex patterns.

SYNTAX

```
name: str = Field(min_length=3)
```

EXAMPLE

```
name: str = Field(min_length=3, max_length=50)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A bank counter checking that the PAN number format is right; Field constraints state exact rules.

தமிழ்

வங்கி கவுண்டர் PAN எண் வடிவம் சரியா என்று சரிபார்ப்பது போல Field constraints சரியான விதிகளைச் சொல்கின்றன.

PROS

- Declarative rules
- Shown in docs

CONS

- Complex rules need validators

ADVANCED / NEW FEATURES

- `Annotated` constraints
- Pydantic v2 validators

08. Dependency Injection

DEFINITION

`Depends` shares logic (DB sessions, auth, pagination) across routes.

SYNTAX

```
def route(db = Depends(get_db)):
```

EXAMPLE

```python
def get_db():
    db = SessionLocal()
    try: yield db
    finally: db.close()

@app.get('/users')
def users(db=Depends(get_db)): ...
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

The shop supplies a fresh notebook to each clerk; Depends injects shared resources like DB sessions.

தமிழ்

ஒவ்வொரு எழுத்தருக்கும் புதிய நோட்டுப் புத்தகம் வழங்குவது போல Depends DB sessions போன்ற பகிரும் வளங்களை வழங்குகிறது.

PROS

- Reusable and testable
- Auth, pagination in one place

CONS

- Deep dependency trees are hard to follow

ADVANCED / NEW FEATURES

- `Annotated[Type, Depends(...)]` pattern
- Dependencies with `yield` scopes

09. Databases with SQLAlchemy

DEFINITION

Define models, sessions and CRUD functions; use Alembic for migrations (or SQLModel).

SYNTAX

```python
class User(Base):
    __tablename__ = 'users'
```

EXAMPLE

```python
class User(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True)
    name = Column(String)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop ledger with columns for each field; SQLAlchemy maps Python classes to tables.

தமிழ்

ஒவ்வொரு புலத்துக்கும் நெடுவரிசை உள்ள கணக்குப் பதிவேடு போல SQLAlchemy Python classes-ஐ tables-உடன் இணைக்கிறது.

PROS

- Powerful ORM
- Migrations with Alembic

CONS

- Session management complexity

ADVANCED / NEW FEATURES

- SQLModel combines Pydantic + SQLAlchemy
- SQLAlchemy 2.0 typing

10. Routers and Project Structure

DEFINITION

Split code with `APIRouter`: routers, schemas, models, services.

SYNTAX

```
router = APIRouter(prefix='/users')
```

EXAMPLE

```
router = APIRouter(prefix='/users', tags=['users'])
app.include_router(router)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Separate counters for billing, returns and enquiries; routers split a big API into parts.

தமிழ்

பில்லிங், திருப்பம், விசாரணைக்கு தனி கவுண்டர்கள் போல routers பெரிய API-ஐ பகுதிகளாகப் பிரிக்கின்றன.

PROS

- Organised code
- Reusable modules

CONS

- Too many layers for tiny apps

ADVANCED / NEW FEATURES

- Router-level dependencies

11. Async Endpoints

DEFINITION

Use `async def` with async libraries (httpx, asyncpg) for high concurrency; plain `def` runs in a thread pool.

SYNTAX

```
async def f(): await ...
```

EXAMPLE

```
@app.get('/data')
async def data():
    async with httpx.AsyncClient() as c:
        return (await c.get('https://api.example.com')).json()
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A waiter serving many tables while food cooks instead of waiting at one table; async endpoints handle many requests together.

தமிழ்

உணவு தயாராகும்போது ஒரு மேசையில் காத்திருக்காமல் பல மேசைகளைக் கவனிக்கும் சர்வர் போல async endpoints பல கோரிக்கைகளை ஒரே நேரத்தில் கையாளும்.

PROS

- High concurrency
- Efficient I/O

CONS

- Blocking calls inside async stall everything

ADVANCED / NEW FEATURES

- Async SQLAlchemy
- Free-threaded Python experiments

12. Background Tasks

DEFINITION

Run work after returning the response (emails, logs).

SYNTAX

```
bg.add_task(send_email, addr)
```

EXAMPLE

```
@app.post('/signup')
def signup(bg: BackgroundTasks):
    bg.add_task(send_email, 'a@b.com')
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Telling the customer 'order received' and packing parcels afterwards; background tasks run after the response.

தமிழ்

'ஆர்டர் பெறப்பட்டது' என்று சொல்லி பிறகு பார்சல் கட்டுவது போல background tasks response-க்குப் பிறகு இயங்கும்.

PROS

- Quick responses
- Simple API

CONS

- Not durable (use Celery/ARQ for heavy jobs)

ADVANCED / NEW FEATURES

- Task queues with ARQ/Celery/Dramatiq

Advanced Level

13. Authentication: OAuth2 and JWT

DEFINITION

Password hashing (bcrypt), OAuth2PasswordBearer and JWT tokens; protect routes with dependencies.

SYNTAX

```
OAuth2PasswordBearer(tokenUrl='token')
```

EXAMPLE

```
oauth2 = OAuth2PasswordBearer(tokenUrl='token')
@app.get('/me')
def me(token: str = Depends(oauth2)): ...
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A hostel entry pass checked at each gate; JWT tokens prove identity on each request.

தமிழ்

ஒவ்வொரு வாயிலிலும் சோதிக்கப்படும் விடுதி அனுமதிச் சீட்டு போல JWT tokens ஒவ்வொரு request-இலும் அடையாளத்தை நிரூபிக்கின்றன.

PROS

- Standard OAuth2
- Docs show Authorize button

CONS

- Must secure secrets and hashing yourself

ADVANCED / NEW FEATURES

- `pwdlib`/Argon2 recommended over passlib
- PyJWT

14. Middleware and CORS

DEFINITION

Add cross-cutting behaviour and allow browser clients.

SYNTAX

```
app.add_middleware(CORSMiddleware, ...)
```

EXAMPLE

```
app.add_middleware(CORSMiddleware, allow_origins=['https://myapp.com'], allow_methods=['*'])
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A gatekeeper who allows only invited houses; CORS controls which websites call your API.

தமிழ்

அழைக்கப்பட்ட வீடுகளை மட்டும் அனுமதிக்கும் வாயிற்காவலர் போல CORS எந்த வலைத்தளங்கள் API-ஐ அழைக்கலாம் என்பதைக் கட்டுப்படுத்தும்.

PROS

- Browser security
- Easy config

CONS

- Misconfig breaks frontends

ADVANCED / NEW FEATURES

- Pure ASGI middleware
- Trusted host middleware

15. File Upload and Streaming

DEFINITION

Use `UploadFile` for uploads and `StreamingResponse` for large downloads.

SYNTAX

```
async def up(file: UploadFile):
```

EXAMPLE

```
@app.post('/upload')
async def up(file: UploadFile): return {'name': file.filename}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Parcels received and weighed before storing; UploadFile streams files safely.

தமிழ்

பெற்ற பார்சல்கள் எடை பார்க்கப்பட்டபின் சேமிக்கப்படுவது போல UploadFile கோப்புகளை பாதுகாப்பாக stream செய்கிறது.

PROS

- Spooled temp files
- Streaming responses

CONS

- Need size limits

ADVANCED / NEW FEATURES

- Streaming uploads to S3

16. WebSockets

DEFINITION

Real-time communication with `@app.websocket`.

SYNTAX

```
@app.websocket('/ws')
```

EXAMPLE

```
@app.websocket('/ws')
async def ws(sock: WebSocket):
    await sock.accept()
    await sock.send_text('hi')
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A phone call rather than letters: both sides talk any time; WebSockets enable real-time updates.

தமிழ்

கடிதங்களுக்குப் பதிலாக தொலைபேசி அழைப்பு போல WebSockets நேரடி மாற்றங்களைத் தருகின்றன.

PROS

- Real-time features
- Easy API

CONS

- Scaling stateful connections

ADVANCED / NEW FEATURES

- Server-Sent Events (SSE) support in newer versions

17. Testing

DEFINITION

Use `TestClient` with pytest; override dependencies for tests.

SYNTAX

```
client = TestClient(app)
```

EXAMPLE

```
client = TestClient(app)
assert client.get('/').status_code == 200
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A trial customer visits your shop to test the service; TestClient calls the API without a server.

தமிழ்

சேவையைச் சோதிக்க மாதிரி வாடிக்கையாளர் வருவது போல TestClient server இல்லாமல் API-ஐ அழைக்கிறது.

PROS

- Fast tests
- Dependency overrides

CONS

- Async tests need setup

ADVANCED / NEW FEATURES

- `httpx.AsyncClient` with ASGITransport

18. Settings and Configuration

DEFINITION

Use `pydantic-settings` to read environment variables safely.

SYNTAX

```
class Settings(BaseSettings):
```

EXAMPLE

```
class Settings(BaseSettings):
    database_url: str
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Shop keys kept in a locker, not on the counter; settings read secrets from environment variables.

தமிழ்

கடைச் சாவிகளை கவுண்டரில் வைக்காமல் லாக்கரில் வைப்பது போல settings ரகசியங்களை environment variables-இலிருந்து படிக்கும்.

PROS

- Typed config
- Per-environment

CONS

- Missing vars fail at start-up (a good thing)

ADVANCED / NEW FEATURES

- `pydantic-settings` package

Master Level

19. Performance and Caching

DEFINITION

Async DB drivers, Redis caching, pagination, connection pooling, running with multiple Uvicorn workers.

SYNTAX

```
fastapi run --workers 4
```

EXAMPLE

```
fastapi run --workers 4
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Opening more counters when the crowd grows; workers, caching and pooling scale an API.

தமிழ்

கூட்டம் அதிகரிக்கும்போது கூடுதல் கவுண்டர்களைத் திறப்பது போல workers, caching, pooling API-ஐ அளவிடுகின்றன.

PROS

- Throughput gains
- Redis caching

CONS

- Operational complexity

ADVANCED / NEW FEATURES

- Uvicorn workers via `--workers`
- Granian server option

20. Docker and Deployment

DEFINITION

Containerise with a slim Python image, deploy behind Nginx/ cloud services, configure health checks.

SYNTAX

```
FROM python:3.12-slim
```

EXAMPLE

```
FROM python:3.12-slim
COPY . /app
RUN pip install -r /app/requirements.txt
CMD ["fastapi", "run", "/app/main.py", "--port", "80"]
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Sending finished goods in standard boxes; Docker images deploy the same everywhere.

தமிழ்

முடிந்த பொருட்களை தரப்படுத்தப்பட்ட பெட்டிகளில் அனுப்புவது போல Docker images எங்கும் ஒரே மாதிரி deploy ஆகின்றன.

PROS

- Reproducible deploys
- Cloud-ready

CONS

- Image size and patching

ADVANCED / NEW FEATURES

- Official FastAPI CLI deployment
- `uv` in Dockerfiles

21. Serving ML Models

DEFINITION

Load a model once at startup (lifespan) and expose prediction endpoints with validated inputs.

SYNTAX

```
@app.post('/predict')
```

EXAMPLE

```
@app.post('/predict')
def predict(x: Features):
    return {'result': model.predict([x.values])[0]}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A doctor who studies old cases (model) and gives a quick opinion; endpoints serve ML predictions.

தமிழ்

பழைய நோயாளிகளைப் படித்து விரைவாகக் கருத்துச் சொல்லும் மருத்துவர் போல endpoints ML கணிப்புகளை வழங்குகின்றன.

PROS

- Simple model serving
- Validated inputs

CONS

- Model loading memory

ADVANCED / NEW FEATURES

- Lifespan loading
- ONNX/vLLM backends

22. Real Project: Blog / Task API with Auth

DEFINITION

Full CRUD API with users, JWT, PostgreSQL, Alembic migrations, tests and Docker.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A small-business backend: users, tasks and auth with a database.

தமிழ்

பயனர்கள், பணிகள், auth, database உடன் சிறு வணிக backend.

PROS

- Good portfolio piece

CONS

- Takes time

ADVANCED / NEW FEATURES

- Use SQLModel, Alembic, pytest, Docker

Interview Question Bank

Q1. Why is FastAPI fast?
It uses Starlette and async I/O, and Pydantic's compiled validation.

Q2. What is Pydantic?
A data validation library that parses and validates data using Python type hints.

Q3. `def` vs `async def` endpoints?
`async def` runs on the event loop (use awaitable libraries); `def` runs in a thread pool.

Q4. What is `Depends`?
FastAPI's dependency injection for reusable logic like DB sessions and auth.

Q5. How are docs generated?
From type hints and Pydantic models into an OpenAPI schema, shown by Swagger UI and ReDoc.

Q6. How do you handle auth?
OAuth2 with password flow or JWT bearer tokens implemented as dependencies.

Q7. Flask vs FastAPI?
Flask is synchronous and minimal; FastAPI is async-first with built-in validation and docs.

Q8. What is `response_model`?
Filters and validates the response data shape.

Q9. How do you run background jobs?
`BackgroundTasks` for light work; Celery/RQ/ARQ for heavy jobs.

Q10. How do you test FastAPI?
`TestClient` plus pytest, overriding dependencies.

Capstone Project Ideas

1. Todo API with PostgreSQL
2. Blog API with JWT
3. URL shortener
4. ML prediction service
5. Chat app with WebSockets